

MetaState: Persistent Working Memory Enhances Reasoning in Discrete Diffusion Language Models

Anonymous authors

Paper under double-blind review

Abstract

Discrete diffusion language models (dLLMs) generate text by iteratively denoising a masked sequence. However, standard dLLMs condition each denoising step solely on the current hard-masked sequence, while intermediate continuous representations are discarded after sampling and remasking. We term this bottleneck the **Information Island** issue: continuous information remains isolated within individual denoising steps and fails to propagate across the denoising trajectory. This bottleneck is especially harmful for reasoning, which requires intermediate reasoning state to be preserved and updated across many denoising steps. To address this limitation, we introduce **MetaState**, a lightweight recurrent augmentation that equips a frozen dLLM backbone with persistent, fixed-size working memory. MetaState comprises three modules with a shared time conditioner: a cross-attention **Mixer** that reads backbone activations into memory slots, a recurrent **Updater** that integrates information across steps, and a cross-attention **Injector** that writes the updated memory back into the backbone. We train these modules with a dedicated K -step unrolling pipeline to learn multi-step dynamics. MetaState adds only $\sim 0.6\%$ trainable parameters while keeping the backbone frozen, and consistently improves reasoning performance over frozen baselines on mathematical reasoning and code generation benchmarks, with an average gain of 4.5% across all evaluations.

1 Introduction

Autoregressive (AR) language models factorize the joint distribution over sequences into a product of conditional probabilities, producing one token per forward pass (Radford et al., 2018; 2019; Brown et al., 2020). Although this paradigm underlies many recent foundation models, the left-to-right causal structure prevents parallel decoding and limits the use of bidirectional context. Discrete diffusion language models (dLLMs) have recently emerged as a non-autoregressive alternative (Li et al., 2025; Sahoo et al., 2024; Gong et al., 2024). Starting from a fully corrupted sequence, dLLMs iteratively denoise to recover clean text and update arbitrary positions using bidirectional attention (Austin et al., 2021a). When scaled to billions of parameters, dLLMs achieve quality comparable to that of autoregressive models while retaining the advantages of decoding parallelism and generation flexibility, as demonstrated by the LLaDA series (Nie et al., 2025; Zhu et al., 2025a; Bie et al., 2025) and Dream (Ye et al., 2025).

Current standard dLLMs nevertheless share a limitation that we term the Information Island issue. In the diffusion formulation, the inter-step process is Markovian in the discrete

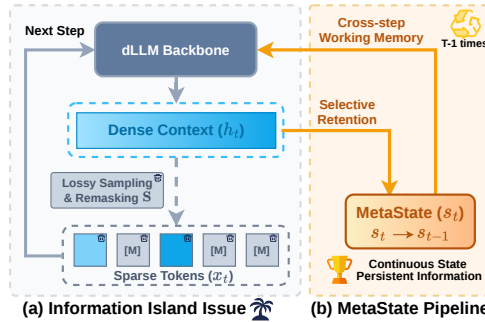


Figure 1: The Information Island issue in discrete diffusion: sampling and remasking compress continuous hidden activations into discrete tokens, imposing a lossy bottleneck between denoising steps. MetaState addresses this issue by maintaining a persistent state across steps.

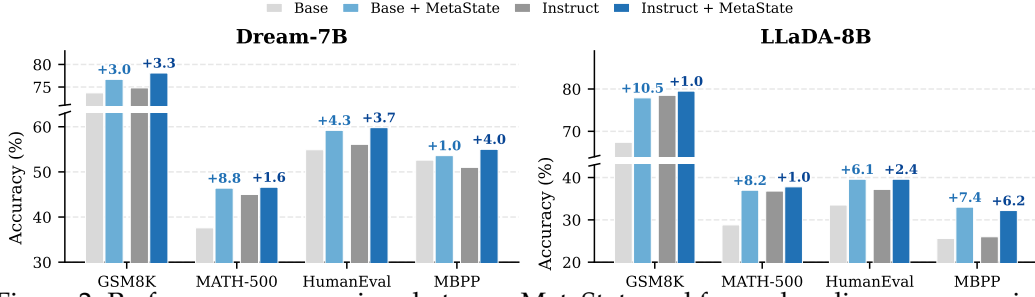


Figure 2: Performance comparison between MetaState and frozen baselines on reasoning benchmarks for LLaDA-8B and Dream-7B in both Instruct and Base versions.

sequence state: each transition is conditioned on the current masked tokens $\mathbf{x}_t$, while the continuous hidden representation $\mathbf{h}_t$ computed at that step is not carried forward explicitly,

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t), \quad \mathbf{x}_{t-1} = \mathcal{S}(\mathbf{h}_t).$$

At each denoising step t , the model computes a high-dimensional hidden representation $\mathbf{h}_t$ that encodes substantially richer information than the discrete tokens passed to the next step. Beyond token-level predictive semantics, $\mathbf{h}_t$ also captures long-range dependencies and global sequence structure information. However, the sampling-and-remasking interface $\mathcal{S}$ maps this rich continuous state to discrete token identities and sparse remasking indicators. This transition discards the continuous information in $\mathbf{h}_t$ and compresses each step’s computation into a sparse discrete sequence. As a result, the next denoising step receives only a highly lossy representation of the information computed at the previous step. We refer to this repeated cross-step information loss as the Information Island issue.

This bottleneck arises at every transition along the denoising trajectory, as illustrated in Fig. 1. In the diffusion process, early high-noise steps often establish coarse global structure, while later low-noise steps refine local details and enforce fine-grained constraints. However, useful inferences made at one step must be reconstructed from the sparse token sequence at later steps. This repeated reconstruction can introduce cross-step drift: intermediate information may be weakened, overwritten, or inconsistently re-derived as denoising proceeds. Such drift is especially harmful for reasoning, where success depends on preserving intermediate computations in multi-step mathematical reasoning and maintaining global program constraints such as variable scope and control flow across long denoising trajectories.

To address this limitation, we propose MetaState, a lightweight recurrent augmentation that equips a frozen dLLM backbone with persistent working memory across denoising steps. Motivated by evidence that working memory capacity is an important factor in language model reasoning (Zhang et al., 2024), MetaState maintains a compact set of continuous memory slots that persist across denoising steps, thereby adding a parallel information path alongside the standard discrete denoising path. Concretely, three lightweight modules form a recurrent loop around the frozen backbone: a Mixer reads backbone activations into the memory slots, an Updater integrates newly extracted information through gated recurrence, and an Injector writes the updated state back into the backbone’s input embeddings for the next step. A shared time conditioner coordinates all three modules. To train this recurrent memory to retain and update information across steps, we further introduce a dedicated K -step unrolling procedure that backpropagates through the denoising trajectory (§4.2).

In summary, our contributions are as follows:

1. We identify the Information Island issue in discrete diffusion language models, a representational bottleneck in which rich hidden activations are compressed into sparse and discrete tokens at every denoising step, and analyze why this issue is especially harmful for multi-step reasoning.
2. We propose **MetaState**, a backbone-agnostic recurrent augmentation that maintains constant-size persistent working memory throughout the denoising process, along with a K -step unrolling training procedure that enables gradient flow through the multi-step state trajectory.

3. We validate MetaState on two distinct dLLM backbones, LLaDA-8B (Nie et al., 2025) and Dream-7B (Ye et al., 2025), over standard mathematical reasoning and code generation benchmarks (GSM8K, MATH-500, HumanEval, and MBPP), achieving a 4.5% average improvement at negligible parameter cost.

2 Related Work

2.1 Discrete Diffusion LLMs

Discrete diffusion LLMs formulate text generation as a non-autoregressive process by adapting diffusion dynamics to discrete token spaces. D3PM (Austin et al., 2021a) formalized this approach using discrete transition matrices. Subsequent models largely adopt the masked diffusion paradigm: MDLM (Sahoo et al., 2024) derives a variational lower bound and SEDD (Lou et al., 2023) introduces a score entropy objective, enabling masked dLLMs to reach likelihood and perplexity levels comparable to those of autoregressive (AR) models. At the billion-parameter scale, the LLaDA series (Nie et al., 2025; Zhu et al., 2025a;b) and Dream (Ye et al., 2025) demonstrate that dLLMs can match AR quality while enabling parallel decoding and bidirectional attention. Semi-AR methods like BD3-LMs (Arriola et al., 2025) and SDAR (Cheng et al., 2025) combine inter-block autoregression with intra-block parallel diffusion. For cache integration, dLLM-Cache (Liu et al., 2025) targets dual redundancy in prompts and responses via adaptive caching, while Fast-dLLM (Wu et al., 2025) leverages activation similarity for block-wise KV-cache reuse.

2.2 Continuous Diffusion and Latent Reasoning

To fully use continuous context, prior works have proposed modifying the diffusion kernel or the decoding interface. CADD and CANDI (Zheng et al., 2025; Pynadath et al., 2025) couple discrete masking with continuous diffusion, introducing hybrid diffusion kernels that change the underlying diffusion formulation. DCoLT (Huang et al., 2025) designs a trajectory-level reasoning policy. Other methods such as LRD and RCD (Zhu et al., 2025c; Hu et al., 2026) replace hard token sampling with probability mixtures. In continuous image diffusion, Recurrent Interface Networks (Jabri et al., 2022) maintain persistent latent tokens across denoising steps, and Diffusion Forcing (Chen et al., 2024) couples an RNN with the diffusion process. In AR modeling, several latent reasoning methods propagate continuous latent representations to bypass discrete decoding bottlenecks, including Coconut, CODI, Soft Thinking, and SwiReasoning (Hao et al., 2024; Shen et al., 2025; Zhang et al., 2025; Shi et al., 2025). Building on this line of work, LaDiR and STAR-LDM (Kang et al., 2025; Lovelace et al., 2026) use latent diffusion for trajectory planning. However, these techniques only apply to sequential generation and are not directly applicable to the dLLM paradigm. As a result, dLLMs still lack a mechanism to maintain continuous memory across diffusion steps to address the Information Island issue.

3 Preliminaries

We consider the masked diffusion paradigm (Austin et al., 2021a; Ou et al., 2024), which defines a forward masking process that progressively replaces tokens with a special [MASK] token $\mathbf{M}$ and a reverse unmasking process that recovers the original sequence.

Forward process. Given a clean sequence $\mathbf{x}_0 = (x_0^{(1)}, \dots, x_0^{(N)})$ over vocabulary $\mathcal{V}$, the forward process produces a noisy sequence $\mathbf{x}_t$ by independently masking each token with probability $1 - \alpha_t$, where α_t denotes the token retention probability from the discrete diffusion convention (Shi et al., 2024; Nie et al., 2025). In continuous time $t \in (0, 1]$, the schedule decreases from $\alpha_0 = 1$ (fully clean) to $\alpha_1 \approx 0$ (fully masked). Each token is kept unchanged with probability α_t , and is replaced by the mask token $\mathbf{M}$ with probability $1 - \alpha_t$.

Reverse process. For any noise level $t \in (0, 1]$, the model p_θ predicts the clean token at each masked position i where $x_t^{(i)} = \mathbf{M}$, yielding the conditional distribution $p_\theta(x_0^{(i)} | \mathbf{x}_t)$.

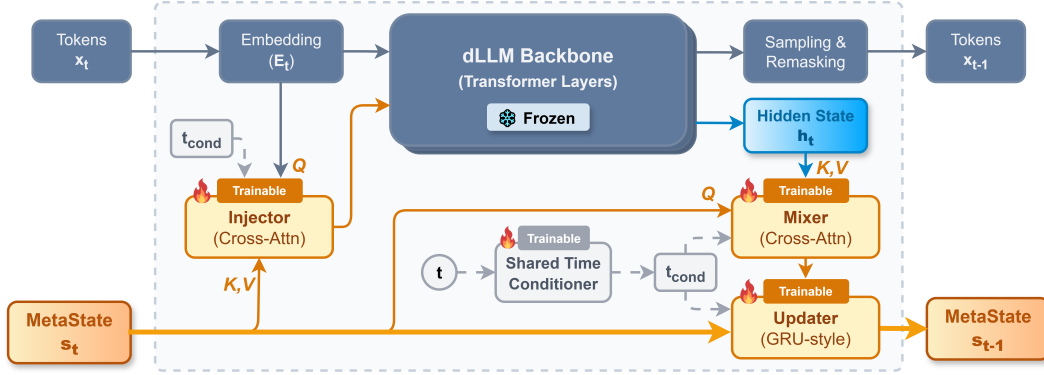


Figure 3: Overview of the MetaState architecture. The three modules (Injector, Mixer, Updater) and the shared time conditioner form a recurrent loop around the frozen backbone, propagating a persistent state across denoising steps.

134 **Training objective.** The training loss is the expected cross-entropy over masked positions:

$$\mathcal{L}_{\text{MDLM}} = \mathbb{E}_{t \sim \mathcal{U}(0,1], \mathbf{x}_0, \mathbf{x}_t \sim q(\mathbf{x}_t | \mathbf{x}_0)} \left[\frac{1}{t} \sum_{i: x_t^{(i)} = \mathbf{M}} -\log p_{\theta}(x_0^{(i)} | \mathbf{x}_t) \right].$$

135 **Inference.** Starting from a fully masked sequence, the time is discretized into T steps. At
 136 each discrete step t , the model samples clean tokens and selectively remasks a subset of
 137 positions according to prediction confidence, yielding a progressively cleaner sequence $\mathbf{x}_{t-1}$.

138 **Information Island issue.** As discussed in §1, this formulation gives rise to the Information
 139 Island issue: the sampling and remasking operator discards continuous hidden activations
 140 at each step (see Appendix A.1 for a detailed analysis).

141 4 Method

142 4.1 MetaState Overview

143 To resolve the Information Island issue, MetaState introduces a continuous memory that
 144 persists across the discrete interface steps. This memory is maintained by three lightweight
 145 modules, the *Injector*, *Mixer*, and *Updater*, coordinated by a shared time conditioner (Figure 3).
 146 All modules operate in reduced-dimensional bottlenecks with a fixed slot count independent
 147 of sequence length. The complete pipeline is given in Algorithm 1 (Appendix A.2).

148 Because the discrete interface between successive denoising steps discards all intermediate
 149 representations, each step has no access to the processing history accumulated by prior steps.
 150 To bridge this gap, we augment the denoising process with a persistent state $\mathbf{s}_t \in \mathbb{R}^{M \times \bar{D}_s}$,
 151 organized as M fixed memory slots of dimension $\bar{D}_s$. This fixed-size design is critical:
 152 it ensures that the memory overhead does not grow with the number of tokens, and it
 153 encourages the network to learn a compact representation of the generation trajectory rather
 154 than simply caching raw activations. The augmented transition becomes:

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{s}_t), \quad \mathbf{s}_{t-1} = g_{\theta}(\mathbf{s}_t, \mathbf{h}_t, t),$$

155 where g_{θ} denotes the state-update function realized by the Mixer and Updater. The state
 156 requires $\mathcal{O}(M\bar{D}_s)$ storage and does not scale with the sequence length N . In addition, each
 157 cross-attention operation is performed with a fixed number of memory slots in a bottleneck
 158 dimension. Thus, the overall overhead is dominated by the cost of the frozen backbone.

159 At each denoising step, the Injector first writes the current state (when the state exists)
 160 into the backbone’s input embeddings using the conditioning feature from the end of the
 161 previous step. After the backbone forward pass, the Mixer reads the final-layer activations
 162 into the memory slots and simultaneously computes a content summary $\bar{\mathbf{h}}_t$, which is
 163 combined with the current timestep t to form the updated conditioning feature $\mathbf{t}_{\text{cond}}$. The

164 Updater then integrates this new context with the existing state via gated recurrence,
165 completing the recurrent loop.

166 4.1.1 Shared Time Conditioner and AdaRMSNorm

167 All three MetaState modules require a shared conditioning signal that captures both the
168 diffusion timestep and the current content of the sequence. A pure timestep embedding is
169 insufficient because the optimal modulation depends on which tokens have already been
170 revealed at each step. We therefore construct a shared time conditioner from a sinusoidal
171 embedding (Vaswani et al., 2017) followed by an MLP, with a zero-gated content residual:

$$\mathbf{t}_{\text{cond}} = \text{MLP}(\text{sinusoidal}(t)) + \alpha_g \odot W_c(\text{RMSNorm}(\bar{\mathbf{h}}_t)) \in \mathbb{R}^{d_c},$$

172 where d_c is the output dimension of the time conditioner, $\bar{\mathbf{h}}_t \in \mathbb{R}^{d_m}$ is the mean-pooled
173 content summary derived from down-projected backbone hidden states (computed by the
174 Mixer before cross-attention, §4.1.2), W_c projects to the time dimension d_c , and $\alpha_g \in \mathbb{R}^{d_c}$ is a
175 learnable per-channel gate. The gate is zero-initialized to ensure that the conditioner begins
176 as a pure timestep function and only gradually incorporates content-aware modulation as
177 training proceeds, avoiding unstable early-stage interactions.

178 The conditioning feature $\mathbf{t}_{\text{cond}}$ is consumed by the normalization layer through AdaRM-
179 SNorm, an adaptive variant of RMSNorm (Zhang & Sennrich, 2019). Each AdaRMSNorm
180 layer includes a modulation projection W_{mod} that produces per-channel scale and shift
181 parameters from the conditioning:

$$[\gamma, \beta] = W_{\text{mod}}(\mathbf{t}_{\text{cond}}), \quad \mathcal{N}(\mathbf{x}, t) = (1 + \gamma) \odot \text{RMSNorm}(\mathbf{x}) + \beta.$$

182 At initialization, AdaRMSNorm reduces to standard RMSNorm, preserving the pretrained
183 behavior of any layer it wraps. We also define a zero-bridge variant $\mathcal{N}_0(\mathbf{x}, t) = \gamma \odot$
184 $\text{RMSNorm}(\mathbf{x}) + \beta$, which outputs nearly $\mathbf{0}$ at initialization. This serves as a zero bridge in
185 the Injector (§4.1.4), ensuring that the augmented model begins as the unmodified backbone.

186 4.1.2 MetaState Mixer

187 The Mixer is designed to convert the variable-length backbone activations $\mathbf{h}_t \in \mathbb{R}^{N \times D}$
188 into a fixed-size representation of M slots, where each slot should capture distinct aspects
189 of the computation rather than collapse into a single pooled summary. We therefore use
190 cross-attention with the state slots as queries, letting each slot selectively read the most
191 informative tokens. To keep the module lightweight, the cross-attention operates in a d_m -
192 dimensional bottleneck, and an up-projection recovers the full state dimension D_s . Before
193 the state representations enter the bottleneck, a slot self-attention layer Attn_{GQA} with plain
194 RMSNorm enables inter-slot coordination in the full D_s space, encouraging different slots to
195 specialize and avoid redundant reads. Both the state and the hidden representation are then
196 down-projected and time-conditioned within the bottleneck:

$$\mathbf{s}_t^b = \mathcal{N}(W_{\downarrow}^s \mathbf{s}_t, t) \in \mathbb{R}^{M \times d_m}, \quad \mathbf{h}_t^b = \mathcal{N}(W_{\downarrow}^h \mathbf{h}_t, t) \in \mathbb{R}^{N \times d_m}.$$

197 Before cross-attention, the Mixer computes a content summary $\bar{\mathbf{h}}_t = \text{MeanPool}(W_{\downarrow}^h \mathbf{h}_t)$ and
198 passes it to the time conditioner (§4.1.1), so that subsequent normalization layers can adapt to
199 the current sequence content. Cross-attention is then computed with $\mathbf{s}_t^b$ as queries and $\mathbf{h}_t^b$ as
200 keys/values, yielding $\mathbf{a}_t^b = \text{CrossAttn}_{\text{GQA}}(\mathbf{s}_t^b, \mathbf{h}_t^b) \in \mathbb{R}^{M \times d_m}$. An FFN with AdaRMSNorm
201 is then applied, followed by an up-projection to yield the Mixer output $\mathbf{c}_t \in \mathbb{R}^{M \times D_s}$.

202 4.1.3 MetaState Updater

203 The Updater must retain information accumulated over earlier denoising steps while in-
204 corporating new context from the current step. A time-conditioned GRU (Cho et al., 2014)
205 addresses this trade-off directly: its learned update gate provides a per-dimension interpola-
206 tion between the existing state and a candidate update. Given the current state $\mathbf{s}_t \in \mathbb{R}^{M \times D_s}$
207 and the Mixer output $\mathbf{c}_t \in \mathbb{R}^{M \times D_s}$, both inputs are first normalized with time conditioning.

A learnable slot identity embedding $\mathbf{e}_{\text{slot}} \in \mathbb{R}^{M \times D_s}$ is added to the state before normalization so that each slot can learn distinct retention and update behaviors. The complete update rule is:

$$\begin{aligned}\bar{\mathbf{s}}_t &= \mathcal{N}(\mathbf{s}_t + \mathbf{e}_{\text{slot}}, t), \quad \bar{\mathbf{c}}_t = \mathcal{N}(\mathbf{c}_t, t), \\ \mathbf{z}_t, \mathbf{r}_t &= \sigma(W_g([\bar{\mathbf{s}}_t \parallel \bar{\mathbf{c}}_t])), \quad \tilde{\mathbf{s}}_t = \tanh(W_s([\mathbf{r}_t \odot \bar{\mathbf{s}}_t \parallel \bar{\mathbf{c}}_t])), \\ \mathbf{s}_{t-1} &= (1 - \mathbf{z}_t) \odot \mathbf{s}_t + \mathbf{z}_t \odot \tilde{\mathbf{s}}_t.\end{aligned}$$

The update gate $\mathbf{z}_t$ controls how much of each dimension is overwritten, while the reset gate $\mathbf{r}_t$ determines how much of the previous state influences the candidate $\tilde{\mathbf{s}}_t$. The final interpolation ensures a smooth transition between retaining old information and integrating new context. Time modulation enters the GRU exclusively through the AdaRMSNorm layers on $\bar{\mathbf{s}}_t$ and $\bar{\mathbf{c}}_t$, which effectively provides timestep-dependent information.

4.1.4 MetaState Injector

The Injector should write the persistent state back into the backbone without disrupting the backbone’s pretrained capabilities. We therefore realize it as an additive modulation of the input embeddings. Given embeddings $\mathbf{e}_t \in \mathbb{R}^{N \times D}$, we first down-project to a d_b -dimensional bottleneck: $\mathbf{x}_t^b = W_{\downarrow}^e \mathbf{e}_t \in \mathbb{R}^{N \times d_b}$. A self-attention layer Attn_{GQA} with sinusoidal positional encoding and plain RMSNorm then enriches these representations with explicit positional context, enabling the subsequent cross-attention to route slot information to the appropriate sequence positions. The state is down-projected and normalized via $\mathbf{s}_t^b = \mathcal{N}(W_{\downarrow}^s \mathbf{s}_t, t)$, and cross-attention is computed with $\mathbf{x}_t^b$ as queries and $\mathbf{s}_t^b$ as keys/values. The output is added as a residual to $\mathbf{x}_t^b$. A subsequent FFN refines the fused representation, and a zero-bridge layer ($\mathcal{N}_0$) up-projects the result back to the full embedding dimension:

$$\mathbf{x}_t^b \leftarrow \mathbf{x}_t^b + \text{FFN}(\mathcal{N}(\mathbf{x}_t^b, t)), \quad \delta_t = W_{\uparrow} \mathcal{N}_0(\mathbf{x}_t^b, t), \quad \tilde{\mathbf{e}}_t = \mathbf{e}_t + \delta_t.$$

Because $\mathcal{N}_0$ outputs near-zero at initialization, the modulation δ_t vanishes at the start of training, ensuring that the augmented model begins as the unmodified backbone while still allowing all Injector parameters to receive gradients. The final modified embeddings $\tilde{\mathbf{e}}_t$ are fed back into the frozen backbone for the current denoising step.

4.2 Training: K-Step Iterative Unrolling

Standard masked diffusion training samples a single random timestep t per example and optimizes a single-step denoising objective. This approach is inadequate for MetaState, whose persistent state forms a recurrent chain across the full denoising trajectory (§4.1): the modules must learn what information to write into the state, what to retain across steps, and how to adapt the gating behavior across the denoising trajectory. We therefore adopt a multi-step unrolling pipeline with backpropagation through time (BPTT) (Werbos, 2002; Gers et al., 2002) along the state trajectory.

Training Pipeline. Starting from a fully masked input, a warmup forward pass initializes the recurrent state without computing loss. A complete reveal trajectory is then pre-sampled to partition all N_m maskable positions into K batches. At each unrolling step, the model first predicts on the current masked input, and a batch of positions is then revealed via teacher forcing. Let $\mathcal{M} \subseteq \{1, \dots, N\}$ denote the set of maskable positions, and let $N_m = |\mathcal{M}|$ denote the total number of such positions. At unrolling step k , let $\mathcal{M}_k$ denote the set of positions that remain masked, with $\mathcal{M}_1 = \mathcal{M}$ and $\mathcal{M}_k \subset \mathcal{M}_{k-1}$. Let $\mathcal{R}_k \subset \mathcal{M}_k$ denote the set of n_k positions revealed at step k . The sequence $\mathbf{x}^{(k+1)}$ is then obtained by revealing the ground-truth tokens at the positions in $\mathcal{R}_k$, and the masked set is updated as $\mathcal{M}_{k+1} = \mathcal{M}_k \setminus \mathcal{R}_k$. The complete training pipeline is summarized in Algorithm 2 in the Appendix.

Dirichlet trajectory. We pre-sample reveal counts $\mathbf{n} = (n_1, \dots, n_K)$ from a symmetric Dirichlet–Multinomial distribution over the K denoising steps, which partitions the N_m maskable positions into step-wise reveal budgets. A random permutation of maskable positions determines the reveal order, and step k reveals the next n_k positions in that order. The timestep at step k is defined as a normalized masked ratio $t^{(k)} = |\mathcal{M}_k|/N_m \in [0, 1]$.

Table 1: Main results on 4 different benchmarks (generation length 256, block size 32, dual cache). MATH = MATH-500, HE = HumanEval. Δ denotes improvement over the corresponding baseline. **Bold** marks the best result per column within each backbone group.

Model	Dream Backbone (7B)				LLaDA Backbone (8B)				Avg.
	GSM8K	MATH	HE	MBPP	GSM8K	MATH	HE	MBPP	
Base	73.7	37.6	54.9	52.6	67.4	28.8	33.5	25.6	46.8
+ MetaState	76.7	46.4	59.2	53.6	77.9	37.0	39.6	33.0	52.9
Δ vs. Base	+3.0	+8.8	+4.3	+1.0	+10.5	+8.2	+6.1	+7.4	+6.2
Instruct	74.8	45.0	56.1	51.0	78.5	36.8	37.2	26.0	50.7
+ MetaState	78.1	46.6	59.8	55.0	79.5	37.8	39.6	32.2	53.6
Δ vs. Instruct	+3.3	+1.6	+3.7	+4.0	+1.0	+1.0	+2.4	+6.2	+2.9

4.2.1 Loss Function

At each unrolling step k , the objective interpolates between dense supervision over all positions that remain masked and focused supervision over the subset scheduled to be revealed:

$$\mathcal{L}_k = w_k \left[\lambda_d \ell_k^{\text{dense}} + (1 - \lambda_d) \ell_k^{\text{reveal}} \right], \quad w_k = \frac{n_k}{N_m},$$

where $n_k = |\mathcal{R}_k|$ is the number of positions revealed at step k , and N_m is the total number of maskable positions. The dense and reveal losses are defined as

$$\ell_k^{\text{dense}} = \frac{1}{|\mathcal{M}_k|} \sum_{i \in \mathcal{M}_k} -\log p_\theta(x_0^{(i)} | \mathbf{x}^{(k)}), \quad \ell_k^{\text{reveal}} = \frac{1}{n_k} \sum_{i \in \mathcal{R}_k} -\log p_\theta(x_0^{(i)} | \mathbf{x}^{(k)}),$$

where $\mathcal{M}_k$ denotes the set of positions still masked at step k , and $\mathcal{R}_k \subseteq \mathcal{M}_k$ denotes the subset selected to be revealed at that step.

We also include a regularization term to penalize per-slot norms that exceed a threshold τ :

$$\mathcal{R}eg_s = \frac{\lambda_s}{KM} \sum_{k=1}^K \sum_{m=1}^M [\max(0, \|s_m^{(k)}\|_2 - \tau)]^2,$$

where τ leaves the state free below the threshold, and λ_s is the regularization weight. The total loss is the sum of per-step losses $\sum_{k=1}^K \mathcal{L}_k$ and the regularization term $\mathcal{R}eg_s$.

5 Experiments

5.1 Experimental Settings

Models and Datasets. We apply MetaState to two discrete diffusion LLM families, each in Base and Instruct variants: LLaDA-Instruct-8B / LLaDA-Base-8B (Nie et al., 2025) and Dream-v0-Instruct-7B / Dream-v0-Base-7B (Ye et al., 2025). To isolate the effect of the recurrent design, all backbone parameters are frozen throughout training. Only the MetaState components (Mixer, Updater, Injector, and the time-conditioning module) are trained, amounting to approximately 0.6% of each backbone’s total parameters. We train on 50,000 sequences sampled from the Tulu-3 SFT mixture (allenai/tulu-3-sft-mixture) (Lambert et al., 2024), using each model’s native tokenizer and chat template with a maximum sequence length of 1024.

Evaluation Benchmarks. We evaluate on four reasoning benchmarks: GSM8K (Cobbe et al., 2021) (5-shot) and MATH-500 (Lewkowycz et al., 2022) (4-shot) for mathematical reasoning, and HumanEval (Chen et al., 2021) (0-shot) and MBPP (Austin et al., 2021b) (3-shot) for code generation. For code benchmarks, accuracy refers to PASS@1, defined as the fraction of problems passing all unit tests. Following standard dLLM practice, the maximum generation length is set to 256 tokens with a block size of 32 tokens. Results at other generation lengths are shown in Appendix A.7. All evaluations use the KV-cache and parallel decoding of Fast-dLLM (Wu et al., 2025).

Table 2: Compatibility with Soft Diffusion (Zhu et al., 2025c). MetaState and Soft Diffusion target different levels of the pipeline and can be combined. † denotes ‘+ Soft Diffusion’. **Bold** marks the best result per column. Hyperparameter details are provided in Appendix A.5.

Method	<i>Dream-Instruct (7B)</i>				<i>LLaDA-Instruct (8B)</i>				Avg.
	GSM8K	MATH	HE	MBPP	GSM8K	MATH	HE	MBPP	
Instruct baseline	74.8	45.0	56.1	51.0	78.5	36.8	37.2	26.0	50.7
+ Soft Diffusion	79.1	45.8	60.4	52.0	78.7	37.8	37.8	29.0	52.6
+ MetaState	78.1	46.6	59.8	55.0	79.5	37.8	39.6	32.2	53.6
+ MetaState†	79.4	46.4	59.2	55.6	80.3	37.6	40.9	32.4	54.0

5.2 Main Results

Table 1 compares MetaState against both Base and Instruct backbones on four reasoning benchmarks. MetaState consistently improves accuracy over both Base and Instruct variants across all benchmarks and both dLLM families. On Dream, MetaState outperforms Dream-Base on every benchmark, with gains of +3.0 on GSM8K, +8.8 on MATH-500, +4.3 on HumanEval, and +1.0 on MBPP. The same trend holds against the stronger Dream-Instruct baseline, where MetaState improves GSM8K by +3.3, MATH-500 by +1.6, HumanEval by +3.7, and MBPP by +4.0. LLaDA exhibits the same pattern at a larger scale. Relative to LLaDA-Base, MetaState yields gains of +10.5 on GSM8K, +8.2 on MATH-500, +6.1 on HumanEval, and +7.4 on MBPP. Relative to LLaDA-Instruct, the improvements are smaller but remain consistent, with gains of +1.0 on GSM8K, +1.0 on MATH-500, +2.4 on HumanEval, and +6.2 on MBPP. The smaller margins over Instruct backbones are expected, since instruction tuning already strengthens task-following behavior and model capabilities through supervised examples.

These results suggest that persistent working memory is particularly beneficial for tasks that require information to remain stable across long denoising trajectories. In mathematical reasoning, the model must retain intermediate computations and partial conclusions until the final answer is formed. In code generation, it must maintain global structural constraints such as variable scope, control flow, and program-level consistency over many lines of code. Both settings are vulnerable to cross-step drift, and the consistent gains of MetaState across all four benchmarks and two architecturally distinct dLLM families support the view that its benefits arise from mitigating the Information Island issue.

5.3 Compatibility with Soft Diffusion

Both MetaState and Soft Diffusion (Zhu et al., 2025c) improve dLLM decoding but through orthogonal mechanisms. Soft Diffusion directly modifies the original discrete decoding path by replacing hard masked token positions with probability-weighted embedding mixtures. MetaState, in contrast, leaves the discrete path unchanged and instead introduces a parallel persistent memory path that carries continuous information. In this sense, Soft Diffusion refines how token representations are formed within each step, whereas MetaState augments the decoding process with an additional cross-step information channel. Because one modifies the discrete path itself and the other adds a separate recurrent pathway alongside it, the two methods are naturally orthogonal and can be combined. Table 2 evaluates this combination on the Instruct variants of both backbones.

Comparing MetaState and Soft Diffusion individually (rows 2 and 3 of Table 2), MetaState outperforms Soft Diffusion on most benchmarks. Applying Soft Diffusion on top of MetaState brings further improvements, and the combination achieves the strongest overall results, including the best GSM8K accuracy of 80.3 and 79.4 on LLaDA and Dream. These results support the view that the two methods are complementary. The only exception is Dream-HumanEval, where MetaState + Soft Diffusion (59.2) underperforms both MetaState alone (59.8) and Soft Diffusion alone (60.4). We attribute this to two factors. First, HumanEval contains only 164 problems, so small differences in accuracy are inherently noisy. Second, the Injector is trained with pure mask embeddings as input, whereas Soft Diffusion

Table 3: Ablation studies. Each row modifies one component while keeping the rest of MetaState unchanged. The MLP variants are parameter-matched. **Bold**: full MetaState model. Gray: original backbone without MetaState (from Table 1).

Model Variants	Dream-Instruct (7B)				LLaDA-Instruct (8B)				Avg.
	GSM8K	MATH	HE	MBPP	GSM8K	MATH	HE	MBPP	
<i>Backbone only</i>	74.8	45.0	56.1	51.0	78.5	36.8	37.2	26.0	50.7
MetaState (full)	78.1	46.6	59.8	55.0	79.5	37.8	39.6	32.2	53.6
w/o recurrence	77.6	45.0	61.0	53.4	62.6	32.6	23.8	24.6	47.6
w/o BPTT	76.7	43.4	59.2	53.2	78.2	34.4	38.4	33.0	52.1
w/o time cond.	77.2	44.8	57.9	54.6	78.9	36.2	39.0	31.8	52.6
MLP Injector	77.2	41.2	54.3	52.2	77.7	36.2	38.4	30.8	51.0
MLP Mixer	76.9	42.4	56.7	54.2	74.4	33.8	33.5	32.2	50.5

replaces these mask inputs with probability-weighted embedding mixtures, introducing an input distribution shift that may not be fully aligned with the Injector’s behavior. Further details on Soft Diffusion hyperparameters are provided in Appendix A.5.

5.4 Ablation Studies

Table 3 presents ablation results for each MetaState component on both Dream-Instruct and LLaDA-Instruct under the same 50k-sample training setup.

Recurrence. Zeroing the previous state at every denoising step (w/o recurrence) has asymmetric effects across backbones: on LLaDA, performance drops substantially below the backbone-only baseline (−15.9 on GSM8K and −13.4 on HumanEval), indicating that the Injector without the recurrent state becomes harmful. On Dream, in contrast, the effect is minimal, and the variant still remains above the Dream backbone on three of four benchmarks. This asymmetry suggests that Dream retains stronger per-step coherence during denoising, whereas LLaDA depends more heavily on the recurrent memory channel to preserve information across steps. Detaching the state between denoising steps (w/o BPTT), which preserves accumulation but blocks gradient flow through the unrolled trajectory, consistently degrades both backbones, with the largest drops on MATH-500 (−3.2 for Dream, −3.4 for LLaDA), showing that BPTT is important for learning useful state dynamics.

Architecture. Replacing the attention-based Injector with a parameter-matched MLP (MLP Injector) that flattens the slot state and broadcasts a uniform bias removes position-aware injection while preserving a path from memory to the token representations. This leads to clear degradation on both backbones, with particularly large drops on Dream (−5.5 on HumanEval and −5.4 on MATH-500). Replacing the attention-based Mixer with a parameter-matched MLP (MLP Mixer) that average-pools backbone hidden states before projection likewise hurts performance consistently, and is especially damaging on LLaDA (−5.1 on GSM8K and −6.1 on HumanEval). Together, these results show that both token-selective reading in the Mixer and position-aware writing in the Injector are important. Removing explicit time conditioning by zeroing the SharedTimeConditioner (w/o time cond.) produces the smallest drops across benchmarks, suggesting that while timestep information is beneficial, the recurrent gating and attention mechanisms can partially infer denoising progress even without explicit conditioning.

6 Conclusion

We presented MetaState, a lightweight recurrent augmentation together with a dedicated training pipeline that equips frozen discrete diffusion LLM backbones with a persistent, fixed-size working memory across denoising steps, thereby addressing the Information Island issue. On frozen LLaDA-8B and Dream-7B backbones, MetaState adds only approximately 0.6% trainable parameters and yields consistent improvements on both mathematical reasoning and code generation benchmarks, demonstrating that persistent cross-step working memory is an effective mechanism for improving reasoning performance in dLLMs.

References

- Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025.
- Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34:17981–17993, 2021a.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021b.
- Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, et al. Llada2. 0: Scaling up diffusion language models to 100b. *arXiv preprint arXiv:2512.15745*, 2025.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Boyuan Chen, Diego Martí Monsó, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. *Advances in Neural Information Processing Systems*, 37:24081–24125, 2024.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebggen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. 2021.
- Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, et al. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.
- Kyunghyun Cho, Bart Van Merriënboer, Çağlar Gulçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp. 1724–1734, 2014.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Felix A Gers, Nicol N Schraudolph, and Jürgen Schmidhuber. Learning precise timing with lstm recurrent networks. *Journal of machine learning research*, 3(Aug):115–143, 2002.
- Shansan Gong, Shivam Agarwal, Yizhe Zhang, Jiacheng Ye, Lin Zheng, Mukai Li, Chenxin An, Peilin Zhao, Wei Bi, Jiawei Han, et al. Scaling diffusion language models via adaptation from autoregressive models. *arXiv preprint arXiv:2410.17891*, 2024.

- 413 Shibo Hao, Sainbayar Sukhbaatar, Dijia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuan-
414 dong Tian. Training large language models to reason in a continuous latent space. *arXiv*
415 *preprint arXiv:2412.06769*, 2024.
- 416 Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang,
417 Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models.
418 *Iclr*, 1(2):3, 2022.
- 419 Yuezhou Hu, Harman Singh, Monishwaran Maheswaran, Haocheng Xi, Coleman Hooper,
420 Jintao Zhang, Aditya Tomar, Michael W Mahoney, Sewon Min, Mehrdad Farajtabar, et al.
421 Residual context diffusion language models. *arXiv preprint arXiv:2601.22954*, 2026.
- 422 Zemin Huang, Zhiyang Chen, Zijun Wang, Tiancheng Li, and Guo-Jun Qi. Reinforcing
423 the diffusion chain of lateral thought with diffusion language models. *arXiv preprint*
424 *arXiv:2505.10446*, 2025.
- 425 Allan Jabri, David Fleet, and Ting Chen. Scalable adaptive computation for iterative
426 generation. *arXiv preprint arXiv:2212.11972*, 2022.
- 427 Haoqiang Kang, Yizhe Zhang, Nikki Lijing Kuang, Nicklas Majamaki, Navdeep Jaitly, Yi-An
428 Ma, and Lianhui Qin. Ladir: Latent diffusion enhances llms for text reasoning. *arXiv*
429 *preprint arXiv:2510.04573*, 2025.
- 430 Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze
431 Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu,
432 Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind
433 Tafford, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi,
434 and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training.
435 2024.
- 436 Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski,
437 Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al.
438 Solving quantitative reasoning problems with language models. *Advances in neural*
439 *information processing systems*, 35:3843–3857, 2022.
- 440 Tianyi Li, Mingda Chen, Bowei Guo, and Zhiqiang Shen. A survey on diffusion language
441 models. *arXiv preprint arXiv:2508.10875*, 2025.
- 442 Zhiyuan Liu, Yicun Yang, Yaojie Zhang, Junjie Chen, Chang Zou, Qingyuan Wei, Shaobo
443 Wang, and Linfeng Zhang. dlm-cache: Accelerating diffusion large language models
444 with adaptive caching. *arXiv preprint arXiv:2506.06295*, 2025.
- 445 Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating
446 the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023.
- 447 Justin Lovelace, Christian Belardi, Sofian Zalouk, Adhitya Polavaram, Srivatsa Kundurthy,
448 and Kilian Q Weinberger. Stop-think-autoregress: Language modeling with latent diffu-
449 sion planning. *arXiv preprint arXiv:2602.20528*, 2026.
- 450 Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai
451 Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint*
452 *arXiv:2502.09992*, 2025.
- 453 Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan
454 Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean
455 data. *arXiv preprint arXiv:2406.03736*, 2024.
- 456 Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. Candi: Hybrid discrete-continuous diffusion
457 models. *arXiv preprint arXiv:2510.22510*, 2025.
- 458 Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language
459 understanding by generative pre-training. 2018.

- 460 Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al.
461 Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- 462 Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory
463 optimizations toward training trillion parameter models. In *SC20: international conference*
464 *for high performance computing, networking, storage and analysis*, pp. 1–16. IEEE, 2020.
- 465 Subham Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin
466 Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion
467 language models. *Advances in Neural Information Processing Systems*, 37:130136–130184,
468 2024.
- 469 Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. Codi:
470 Compressing chain-of-thought into continuous space via self-distillation. In *Proceedings of*
471 *the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 677–693, 2025.
- 472 Dachuan Shi, Abedelkadir Asi, Keying Li, Xiangchi Yuan, Leyan Pan, Wenke Lee, and Wen
473 Xiao. Swireasoning: Switch-thinking in latent and explicit for pareto-superior reasoning
474 llms. *arXiv preprint arXiv:2510.05069*, 2025.
- 475 Jiabin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and
476 generalized masked diffusion for discrete data. *Advances in neural information processing*
477 *systems*, 37:103131–103167, 2024.
- 478 Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez,
479 Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural informa-*
480 *tion processing systems*, 30, 2017.
- 481 Wen Wang, Bozhen Fang, Chenchen Jing, Yongliang Shen, Yangyi Shen, Qiuyu Wang, Hao
482 Ouyang, Hao Chen, and Chunhua Shen. Time is a feature: Exploiting temporal dynamics
483 in diffusion language models. *arXiv preprint arXiv:2508.09138*, 2025.
- 484 Paul J Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of*
485 *the IEEE*, 78(10):1550–1560, 2002.
- 486 Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo,
487 Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling
488 kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.
- 489 Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Ling-
490 peng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*,
491 2025.
- 492 Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in neural*
493 *information processing systems*, 32, 2019.
- 494 Chunhui Zhang, Yiren Jian, Zhongyu Ouyang, and Soroush Vosoughi. Working memory
495 identifies reasoning limits in language models. In *Proceedings of the 2024 Conference on*
496 *Empirical Methods in Natural Language Processing*, pp. 16896–16922, 2024.
- 497 Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, Shuohang Wang, Yelong
498 Shen, and Xin Eric Wang. Soft thinking: Unlocking the reasoning potential of llms in
499 continuous concept space. *arXiv preprint arXiv:2505.15778*, 2025.
- 500 Huangjie Zheng, Shansan Gong, Ruixiang Zhang, Tianrong Chen, Jiatao Gu, Mingyuan
501 Zhou, Navdeep Jaitly, and Yizhe Zhang. Continuously augmented discrete diffusion
502 model for categorical generative modeling. *arXiv preprint arXiv:2510.01329*, 2025.
- 503 Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou,
504 Jianfei Chen, Yankai Lin, Ji-Rong Wen, et al. Llada 1.5: Variance-reduced preference
505 optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025a.

- 506 Fengqi Zhu, Zebin You, Yipeng Xing, Zenan Huang, Lin Liu, Yihong Zhuang, Guoshan Lu,
507 Kangyu Wang, Xudong Wang, Lanning Wei, et al. Llada-moe: A sparse moe diffusion
508 language model. *arXiv preprint arXiv:2509.24389*, 2025b.
- 509 Qinglin Zhu, Yizhen Yao, Runcong Zhao, Yanzheng Xiang, Amrutha Saseendran, Chen Jin,
510 Philip Teare, Bin Liang, Yulan He, and Lin Gui. Latent refinement decoding: Enhancing
511 diffusion-based language models by refining belief states. *arXiv preprint arXiv:2510.11052*,
512 2025c.

A Appendix

A.1 The Information Island Issue

The Information Island issue arises from the representational gap between the continuous hidden state $\mathbf{h}_t$ computed within each denoising step and the discrete sequence $\mathbf{x}_{t-1}$ passed to the next step. This gap is inherent to the Markovian formulation of dLLM decoding: each inter-step transition conditions solely on the current discrete state $\mathbf{x}_t$, while the continuous representation $\mathbf{h}_t$ is not carried forward explicitly. On the one hand, $\mathbf{h}_t$ encodes substantially richer information than what the discrete tokens can carry: beyond token-level predictive distributions, it captures long-range dependencies, partial reasoning information, and global structural constraints over the sequence. On the other hand, the next-step input $\mathbf{x}_{t-1} = \mathcal{S}(\mathbf{h}_t)$ is produced by the sampling-and-remasking operator $\mathcal{S}$, which retains only discrete token identities at a sparse subset of positions and discards all remaining continuous context embedded in $\mathbf{h}_t$. This persistent gap at every transition along the denoising trajectory is what we term the Information Island issue.

This bottleneck degrades dLLMs in several related ways. First, useful information computed at one step cannot be directly reused at the next step, and the subsequent step can only re-derive it from a sparse, partially masked sequence that carries no trace of the earlier computation. Second, because this re-derivation occurs under changing noise levels and mask patterns, the trajectory can drift: information that is correct at one step may be weakened, overwritten, or inconsistently re-derived at later steps. Third, the problem is especially severe for reasoning. Tasks such as multi-step mathematics and code generation require the model to make use of intermediate computations. Without an explicit cross-step memory mechanism, these intermediate results are repeatedly exposed to the lossy discrete interface, making it difficult for them to make a useful contribution to later steps.

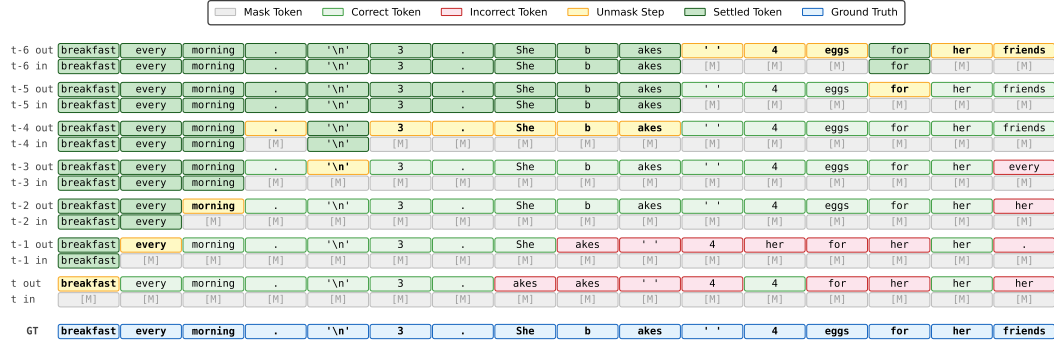


Figure 4: A step-by-step denoising trajectory. Each denoising step shows both the full argmax model prediction (out) and the remasked sequence (in) that is actually passed to the next step.

A step-by-step denoising example. Figure 4 makes this failure mode concrete by showing, at each denoising step, both (i) the full decoded prediction before remasking denoted by $\hat{\mathbf{x}}_t$, and (ii) the remasked sequence input $\mathbf{x}_{t-1}$ that is actually passed to the next step. The key observation is that correct information can already appear in $\hat{\mathbf{x}}_t$ several steps before generation is finalized, yet much of it is lost after remasking and never reaches subsequent steps. Each subsequent step receives only a partial discrete snapshot and must re-derive the missing relations from scratch. In the example, some correct tokens or reasoning fragments appear early, but because they are not preserved across the discrete interface, later steps fail to reuse them and may instead drift toward an inconsistent continuation.

Trajectory-level evidence from PASS@1 and EVERPASS@1. The Information Island issue not only discards useful intermediate representations, but can also degrade final generation quality, as correct tokens produced at earlier steps may be overwritten with incorrect ones after passing through the discrete sampling-and-remasking interface. Following recent analyses of temporal dynamics in dLLMs (Wang et al., 2025), Figure 5 compares PASS@1 with EVERPASS@1, where EVERPASS@1 counts an example as successful if any intermediate full prediction along the denoising trajectory is correct.

553 In our evaluation, EVERPASS@1
 554 remains significantly higher than
 555 PASS@1, indicating that many cor-
 556 rect results are already present
 557 at intermediate denoising steps
 558 but are lost through the subse-
 559 quent discrete remasking opera-
 560 tion. In other words, the model
 561 often “knows” the right answer
 562 somewhere along the trajectory,
 563 yet fails to preserve that informa-
 564 tion through subsequent sampling
 565 and remasking. Together with
 566 the case study in Figure 4, these
 567 statistics show that the Informa-
 568 tion Island issue is a common fail-
 569 ure mode in discrete diffusion lan-
 570 guage models.

571 These observations motivate MetaState, which introduces a persistent continuous working
 572 memory that carries useful intermediate information across steps and tackles the problems
 573 mentioned above.

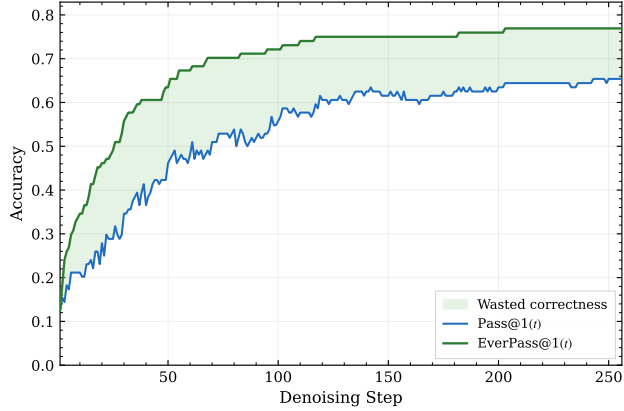


Figure 5: Comparison of $PASS@1(t)$ and EVER- $PASS@1(t)$ across denoising steps on GSM8K with LLaDA-Instruct-8B.

574 A.2 Pseudocode for MetaState

575 Algorithm 1 details the single denoising step procedure of MetaState, and Algorithm 2
576 summarizes the full K -step iterative unrolling training procedure.

Algorithm 1 MetaState: Single Denoising Step

Require: Noisy sequence $\mathbf{x}_t$, persistent state $\mathbf{s}_t$ (or None), previous time conditioning $\mathbf{t}_{\text{cond}}$ (or None), timestep t
Ensure: Logits $\hat{\mathbf{x}}_t$, updated state $\mathbf{s}_{t-1}$, time conditioning $\mathbf{t}_{\text{cond}}$

- 1: $\mathbf{e}_t \leftarrow \text{Embed}(\mathbf{x}_t)$
- 2: **if** $\mathbf{s}_t = \text{None}$ **then** ▷ Warmup step
- 3: $\mathbf{s}_t \leftarrow \mathbf{s}_0$ ▷ Learnable init
- 4: $\tilde{\mathbf{e}}_t \leftarrow \mathbf{e}_t$ ▷ Skip injection
- 5: **else**
- 6: $\tilde{\mathbf{e}}_t \leftarrow \text{Injector}(\mathbf{e}_t, \mathbf{s}_t, \mathbf{t}_{\text{cond}})$ ▷ Additive modulation
- 7: **end if**
- 8: $(\mathbf{h}_t, \hat{\mathbf{x}}_t) \leftarrow p_\theta(\tilde{\mathbf{e}}_t)$ ▷ Frozen backbone
- 9: $\tilde{\mathbf{h}}_t \leftarrow \text{MeanPool}(W_\downarrow^h \mathbf{h}_t)$ ▷ Content summary
- 10: $\mathbf{t}_{\text{cond}} \leftarrow \text{TimeCond}(t, \tilde{\mathbf{h}}_t)$ ▷ Content-aware
- 11: $\mathbf{c}_t \leftarrow \text{Mixer}(\mathbf{s}_t, \mathbf{h}_t, \mathbf{t}_{\text{cond}})$ ▷ Read from $\mathbf{h}_t$
- 12: $\mathbf{s}_{t-1} \leftarrow \text{Updater}(\mathbf{s}_t, \mathbf{c}_t, \mathbf{t}_{\text{cond}})$ ▷ Update state
- 13: **return** $(\hat{\mathbf{x}}_t, \mathbf{s}_{t-1}, \mathbf{t}_{\text{cond}})$ ▷ For next step

Algorithm 2 MetaState Training

Require: Ground truth $\mathbf{x}_0$, maskable positions $\mathcal{M}$, K steps

- 1: $\mathbf{x} \leftarrow \text{mask all positions in } \mathcal{M} \text{ with [MASK]}$
- 2: $(\text{logits}, \mathbf{s}, \mathbf{t}_{\text{cond}}) \leftarrow \text{Forward}(\mathbf{x}, \text{state}=\text{None}, \mathbf{t}_{\text{cond}}=\text{None}, t=1.0)$ ▷ Warmup (Alg. 1)
- 3: Sample reveal counts $n_1, \dots, n_K \sim \text{Dir-Multi}$
- 4: Sample random reveal ranks for $\mathcal{M}$
- 5: $\mathcal{L} \leftarrow 0$
- 6: **for** $k = 1$ to K **do**
- 7: $t \leftarrow |\text{still masked}| / N_m$ ▷ Continuous timestep
- 8: $(\text{logits}, \mathbf{s}, \mathbf{t}_{\text{cond}}) \leftarrow \text{Forward}(\mathbf{x}, \mathbf{s}, \mathbf{t}_{\text{cond}}, t)$ ▷ Alg. 1
- 9: $\mathcal{L} \leftarrow \mathcal{L} + \mathcal{L}_k(\text{logits}, \mathbf{x}_0, \mathcal{M}_k, \mathcal{R}_k)$
- 10: Reveal n_k positions (teacher forcing), update $\mathbf{x}$
- 11: **end for**
- 12: **return** $\mathcal{L} + \lambda_s \text{Reg}_s$

577 A.3 Experimental Details and Hyperparameters

578 A.3.1 Experimental Details.

579 We freeze the backbone and train only the MetaState recurrent components (Mixer, Updater,
580 Injector, and SharedTimeConditioner) with AdamW ($\beta_1=0.9$, $\beta_2=0.95$), a peak learning rate
581 of 2×10^{-5} with cosine decay and 5% linear warmup, and gradient clipping at max norm 1.0.
582 All MetaState weights are initialized from a truncated normal distribution ($\sigma=0.02$), except
583 for the zero-initialized ones. We train for one epoch on 50,000 sequences from the Tulu-3
584 SFT mixture (Lambert et al., 2024), with a maximum sequence length of 1,024 tokens, using
585 bfloat16 mixed precision with DeepSpeed ZeRO-1 (Rajbhandari et al., 2020) on two NVIDIA
586 H200 GPUs. Unless noted otherwise, all hyperparameters are shared across both backbones:
587 $M=64$ memory slots, state dimension $D_s=1024$, bottleneck dimensions $d_m=d_h=768$, and
588 unroll depth $K=4$. Loss is computed only over the response portion, and prompt tokens
589 are excluded from both masking and loss computation. A hinge state-norm regularizer
590 ($\lambda_s=1e-4$, threshold $\tau=1.0$) penalizes per-slot norms that exceed the threshold.

591 During evaluation, Dream performs a single warmup forward pass at $t=1.0$ with $\mathbf{s}=\text{None}$
592 before denoising begins, initializing the recurrent state. Chat-template application varies by
593 backbone: LLaDA-Base evaluations omit the chat template entirely. LLaDA-Instruct applies

594 it for GSM8K and MATH-500 but omits it for HumanEval and MBPP. All Dream variants
 595 (Base and Instruct) apply the backbone’s native chat template. Table 5 lists the remaining
 596 inference hyperparameters.

597 A.3.2 Architecture Hyperparameters.

598 Table 4 summarizes the architectural configuration of all MetaState modules.

Table 4: Architecture hyperparameters for the recurrent modules. All symbols correspond to notation introduced in §4.

Symbol	Description	Value
M	Number of memory slots	64
D_s	State dimension per slot	1024
d_c	Time conditioner output dimension	1024
d_m	Mixer bottleneck dimension	768
d_b	Injector bottleneck dimension	768
n_q	Query heads (Mixer & Injector)	8
n_{kv}	KV heads (GQA)	4
—	SwiGLU FFN expansion ratio	2.0
α_g	Content gate initialization	0 (zero-init)
$\mathcal{N}_0$	Zero-bridge soft bias	10^{-3}

599 A.3.3 Training and Inference Hyperparameters.

600 Table 5 lists the optimization, unrolling, and inference settings shared across both LLaDA
 601 and Dream backbones.

Table 5: Training and inference hyperparameters (shared across LLaDA and Dream backbones).

Parameter	Value
<i>Optimization</i>	
Optimizer	AdamW
Learning rate	2×10^{-5}
LR schedule	cosine, 5% linear warmup
Gradient clipping	max norm 1.0
β_1, β_2	0.9, 0.95
Batch size (effective)	8 (4/GPU $\times$ 2 GPUs)
Epochs	1
Max sequence length	1024
Precision	bfloat16
<i>Unrolling & Loss</i>	
K (unroll steps)	4
λ_d (dense reveal mix)	0.75
λ_s (state norm weight)	10^{-4}
τ (state norm threshold)	1.0
<i>Inference</i>	
Max generation length	256
Block size	32
KV cache mode	dual
Confidence threshold	0.9

602 A.4 Model Parameter Analysis

603 All recurrent modules operate in bottleneck dimensions $d_m=d_b=768$ and memory slot
 604 dimension $D_s=1024$ rather than the backbone hidden size D , which keeps the parameter

count compact. Only three interface projections, the Mixer’s backbone down-projection $W_{\downarrow}^h \in \mathbb{R}^{d_m \times D}$, and the Injector’s input/output projections $W_{\downarrow}^e \in \mathbb{R}^{d_b \times D}$, $W_{\uparrow} \in \mathbb{R}^{D \times d_b}$, depend on the backbone hidden size D . All other parameters are shared the same across backbones. Table 6 provides a per-module breakdown.

Table 6: Per-module parameter breakdown of the MetaState recurrent modules for each backbone ($D_s=1024$, $d_m=d_b=768$, $M=64$). Rows marked † are the only backbone-dependent components.

Module / Component	LLaDA	Dream
SHAREDTIMECONDITIONER	2,102,016	2,102,016
Sinusoidal MLP (256→1024→1024)	1,312,768	1,312,768
Content path (norm + proj + gate)	789,248	789,248
MIXER	17,899,264	17,506,048
Slot self-attention (D_s space, GQA)	3,146,752	3,146,752
Down-projections † ($d_m \times D_s + d_m \times D$)	3,932,160	3,538,944
Cross-attention + AdaRMSNorms	6,494,976	6,494,976
SviGLU FFN + up-projection	4,325,376	4,325,376
UPDATER	10,626,048	10,626,048
AdaRMSNorms ($\times 2$)	4,200,448	4,200,448
GRU projections (gate + candidate)	6,294,528	6,294,528
Learnable s_0 + slot embeddings	131,072	131,072
INJECTOR	20,457,217	19,670,785
Interface projections † ($d_b \times D + D \times d_b$)	6,291,456	5,505,024
Self-attention (RMSNorm, GQA)	1,770,241	1,770,241
State proj + cross-attn + AdaRMSNorms	8,856,576	8,856,576
SviGLU FFN	3,538,944	3,538,944
Total trainable	51,084,545	49,904,897
Frozen backbone	8,056,602,369	7,655,458,049
Overhead	0.63%	0.65%

A.5 Soft Diffusion Hyperparameter Details

Soft Diffusion, introduced in Latent Refinement Decoding (LRD) (Zhu et al., 2025c), replaces the hard remasking at mask positions with a weighted mixture of token embeddings. For each mask position i with sampled token $\hat{x}_i$, the input embedding becomes:

$$\tilde{\mathbf{e}}_i = (1 - r_f) \cdot \mathbf{e}_{\hat{x}_i} + r_f \cdot \sum_v p_v \cdot \mathbf{e}_v,$$

where p_v is the model’s predicted probability for token v (after nucleus filtering with threshold p), $r_f \in [0, 1]$ is the mix ratio factor controlling the mixture weight, and $\mathbf{e}_v$ denotes the token embedding for vocabulary entry v . The two key hyperparameters are the nucleus probability threshold (top- p) and the mix ratio factor (r_f).

Following the practice of LRD (Zhu et al., 2025c), we consider top- $p \in \{0.2, 0.9\}$, since the paper reports that values ≥ 0.2 are effective, and $r_f \in \{0.1, 0.15, 0.2\}$, which spans the effective range $[0.1, 0.2]$ reported in the paper. For each method–backbone–benchmark combination, we evaluate all six (p, r_f) pairs listed above and report the best-performing configuration. Table 7 lists the selected hyperparameters.

Table 7: Best Soft Diffusion hyperparameters (p, r_f) for each method, backbone, and benchmark. All values selected by grid search over $p \in \{0.2, 0.9\}$ and $r_f \in \{0.1, 0.15, 0.2\}$.

Backbone	Method	GSM8K	MATH	HE	MBPP
Dream-Instruct	Soft Diffusion	(0.9, 0.15)	(0.9, 0.1)	(0.9, 0.2)	(0.9, 0.2)
Dream-Instruct	+ MetaState	(0.2, 0.1)	(0.2, 0.2)	(0.9, 0.1)	(0.2, 0.1)
LLaDA-Instruct	Soft Diffusion	(0.2, 0.1)	(0.2, 0.15)	(0.9, 0.2)	(0.2, 0.2)
LLaDA-Instruct	+ MetaState	(0.2, 0.1)	(0.9, 0.2)	(0.2, 0.15)	(0.2, 0.2)

A.6 Comparison with LoRA Fine-Tuning

MetaState and LoRA (Hu et al., 2022) both introduce a small number of trainable parameters on top of a frozen backbone, but they operate at fundamentally different levels. LoRA injects low-rank updates into the backbone’s linear layers, directly modifying the model’s internal representations and thereby its learned inner capabilities. MetaState, by contrast, leaves all backbone weights unchanged and operates entirely at the denoising interface: it reads from the backbone hidden states after each forward pass through the Mixer and writes a lightweight additive signal into the token embeddings before the next pass through the Injector. No gradient flows into the backbone during MetaState training, and the backbone’s per-step predictive behavior is affected only through this external recurrent state channel. In this sense, MetaState introduces cross-step information passing without enhancing the model’s internal capability, whereas LoRA improves single-step capability.

Because the two methods affect different aspects of the generation process, this comparison should not be interpreted as a head-to-head evaluation of fine-tuning methods. Nevertheless, a LoRA baseline under a matched training setup helps rule out an alternative explanation of MetaState’s gains: namely, that the improvements arise primarily from exposure to the Tülu-3 training distribution rather than from the working-memory mechanism itself. To test this possibility, we fine-tune the same Instruct backbones with LoRA ($r=32$, $\alpha=64$, target_modules=all-linear) on the identical 50k Tülu-3 sequences, using the same optimizer, learning-rate schedule, and training budget described in Appendix A.3.1.

Table 8: Comparison with LoRA fine-tuning on Instruct backbones. LoRA ($r=32$, $\alpha=64$, all-linear) is trained on the same 50k Tülu-3 data with matched optimization hyperparameters. † denotes ‘+ Soft Diffusion’. **Bold** marks the best result per column.

Method	<i>Dream-Instruct (7B)</i>		<i>LLaDA-Instruct (8B)</i>		Avg.
	GSM8K	HumanEval	GSM8K	HumanEval	
Instruct	74.8	56.1	78.5	37.2	61.7
+ LoRA	76.0	55.5	79.2	39.0	62.4
+ MetaState	78.1	59.8	79.5	39.6	64.3
+ MetaState [†]	79.4	59.2	80.3	40.9	65.0

Table 8 shows that LoRA underperforms MetaState on every benchmark pair. On Dream-Instruct, LoRA even slightly reduces HumanEval accuracy relative to the original backbone (55.5 vs. 56.1), whereas MetaState improves it to 59.8. On LLaDA-Instruct, LoRA narrows the gap but still remains below MetaState on both GSM8K (79.2 vs. 79.5) and HumanEval (39.0 vs. 39.6). When MetaState is further combined with Soft Diffusion (last row), the margin widens on average across the evaluated benchmarks. Since LoRA directly updates backbone weights and therefore has greater capacity to absorb distributional regularities from the training data than MetaState’s frozen-backbone design, its inferior performance suggests that MetaState’s gains cannot be explained simply by data-domain exposure. Instead, the improvements should be attributed to the persistent working memory introduced by MetaState: by carrying continuous information across denoising steps, MetaState alleviates the Information Island issue (§A.1) in a way that LoRA’s single-step weight adaptation cannot replicate.

A.7 Effect of Generation Length

The main results in Table 1 use a maximum generation length of 256 tokens, following the default inference setting for dLLMs. To assess whether MetaState’s gains extend beyond this standard, we additionally evaluate both backbones with a maximum generation length of 512 tokens. The results are shown in Figure 6.

Across both Dream and LLaDA, MetaState yields consistent improvements at both generation lengths, with gains ranging from +0.1 to +17.2 points. These results indicate that the benefits of persistent working memory are not limited to the default 256-token setting, but

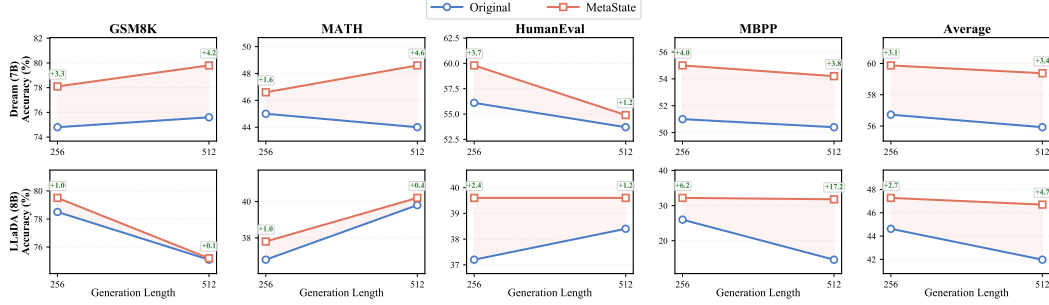


Figure 6: Accuracy comparison between the original backbone and MetaState at generation lengths of 256 and 512 tokens. Numbers above the bars denote the accuracy gain of MetaState over the corresponding backbone. MetaState consistently improves over the original backbone across all benchmarks and generation lengths.

663 remain effective when generation is extended to longer sequences. This trend is consistent
 664 with the Information Island hypothesis (§A.1). In the absence of an explicit cross-step mem-
 665 ory mechanism, longer generation trajectories require the model to repeatedly reconstruct
 666 intermediate context from the current denoising state alone, increasing the risk of losing
 667 higher-level structural information over time. MetaState mitigates this failure mode by
 668 maintaining a recurrent state across denoising steps.

669 A.8 Hyperparameter Sensitivity

670 To evaluate the robustness of MetaState, we vary four hyperparameters individually while
 671 keeping all others fixed at their default values in Table 5. Specifically, we sweep the dense-
 672 reveal loss mixing ratio $\lambda_d \in \{0.60, 0.75, 0.90\}$ (default: 0.75), the number of memory slots
 673 $M \in \{32, 48, 64\}$ (default: 64), the training set size $\in \{30k, 40k, 50k\}$ (default: 50k), and the
 674 unroll depth $K \in \{3, 4, 5\}$ (default: 4). All experiments use the same optimizer, learning-rate
 675 schedule, and evaluation protocol described in Appendix A.3.1, and are conducted on two
 676 NVIDIA A100 GPUs. Figures 7–10 report per-task accuracy on both Dream-Instruct and
 677 LLaDA-Instruct backbones.

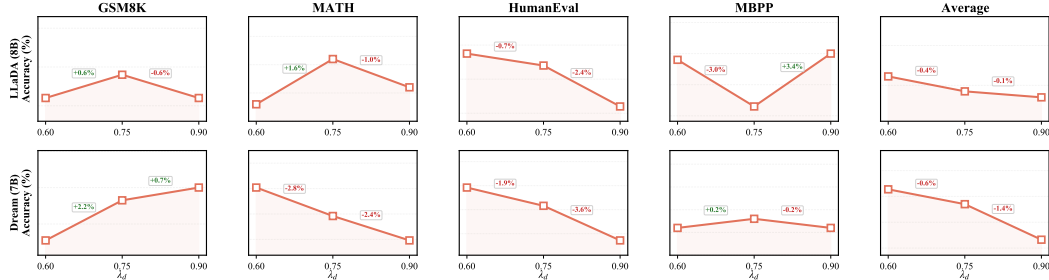


Figure 7: Sensitivity to the dense-reveal loss mixing ratio λ_d . Increasing λ_d from 0.6 to 0.9 yields modest drops in average scores.

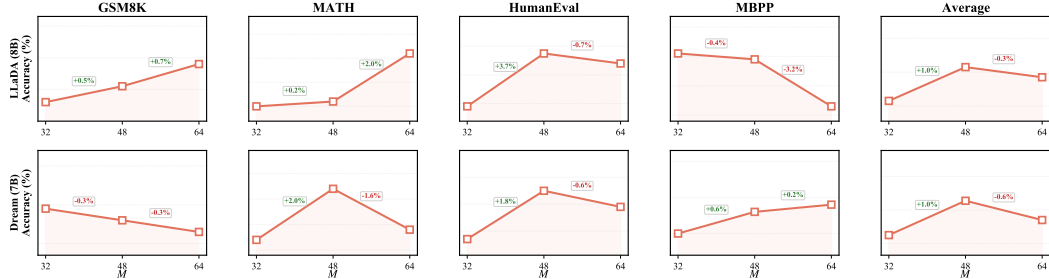


Figure 8: Sensitivity to the number of memory slots M . Increasing M from 32 to 48 yields modest gains on most benchmarks, while further increasing to 64 produces modest drops.



Figure 9: Sensitivity to training data size. MetaState achieves competitive performance even with 30k training examples, and accuracy remains stable as the dataset increases to 50k.

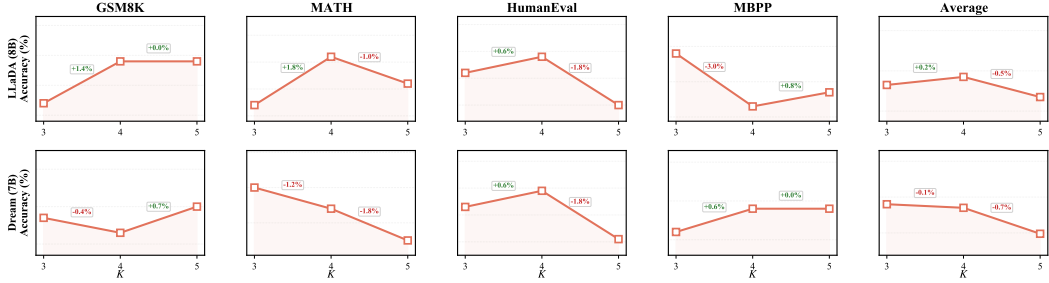


Figure 10: Sensitivity to the unroll depth K . Performance is broadly stable across $K \in \{3, 4, 5\}$, with average accuracy varying by less than one point.

678 Across all four sweeps, performance varies only within a relatively narrow range on both
 679 backbones and across all benchmarks. Overall, these results suggest that MetaState is not
 680 overly sensitive to a single hyperparameter, and that the default setting represents a robust
 681 operating choice rather than a narrowly tuned optimum.

682 A.9 Case Study

683 To provide a qualitative view of how MetaState influences the denoising process, we
 684 present case studies that compare the full denoising trajectories of the original LLaDA-
 685 Instruct and Dream-Instruct backbones with their counterparts augmented by MetaState
 686 on mathematical reasoning examples. All experiments in this section are conducted on
 687 NVIDIA A100 GPUs. The results show a clear qualitative pattern: with MetaState, the
 688 denoising trajectory is more likely to preserve correct intermediate computations, organize
 689 the correct solution structure earlier, and maintain a coherent reasoning path through later
 690 steps. These observations provide evidence that MetaState mitigates the Information Island
 691 issue by carrying forward useful intermediate information that would otherwise be lost
 692 across the discrete sampling-and-remasking interface.

Case Study 1: LLaDA-Instruct (8B) vs. LLaDA-Instruct (8B) + MetaState

Question:

Toula went to the bakery and bought various types of pastries. She bought 3 dozen donuts which cost \$68 per dozen, 2 dozen mini cupcakes which cost \$80 per dozen, and 6 dozen mini cheesecakes for \$55 per dozen. How much was the total cost?
 Please solve this step by step and end with ##### <answer>.

Ground-truth answer: 694

Denoising Timestep $t = 0.80$

LLaDA-Instruct (8B)

Cost of donuts: [MASKx1] 3 [MASKx90]

LLaDA-Instruct (8B) + MetaState

The cost of the donuts is [MASKx90]

Denoising Timestep $t = 0.60$

Cost of donuts: 3 dozen [MASKx1]
 \$68 [MASKx1] dozen [MASKx8]
 mini cupcakes: 2 dozen [MASKx1]
 \$80 [MASKx65]

The cost of the donuts is 3 [MASKx1]
 \$68 = \$204 [MASKx25] mini [MASKx53]

Denoising Timestep $t = 0.40$

Cost of donuts: 3 dozen [MASKx1] \$68
 per dozen = \$204
 Cost of mini cupcakes: 2
 dozen [MASKx1] \$80 per dozen = \$160
 Cost of mini cheesecakes: 6 dozen *
 \$55 per dozen = \$330
 Total cost: [MASKx32]

The cost of the donuts is
 3 [MASKx1] \$68 = \$204 [MASKx15]
 \$160. [MASKx1] The cost of the mini
 cheesecakes is 6 [MASKx1] \$55 = \$330.
 [MASKx36]

Denoising Timestep $t = 0.20$

Cost of donuts: 3 dozen * \$68 per
 dozen = \$204
 Cost of mini cupcakes: 2 dozen * \$80
 per dozen = \$160
 Cost of mini cheesecakes: 6 dozen *
 \$55 per dozen = \$330
 Total cost: \$204 + \$160 + \$330 = \$794
 #### [MASKx6] 9 [MASKx1]

The cost of the donuts is 3 * \$68 =
 \$204 [MASKx6] cupcakes is 2 * \$80 =
 \$160.
 The cost of the mini cheesecakes is 6
 * \$55 = \$330.
 [MASKx31] [MASKx1] 94

Denoising Timestep $t = 0.00$

Cost of donuts: 3 dozen * \$68 per
 dozen = \$204
 Cost of mini cupcakes: 2 dozen * \$80
 per dozen = \$160
 Cost of mini cheesecakes: 6 dozen *
 \$55 per dozen = \$330
 Total cost: \$204 + \$160 + \$330 = \$794
 #### 794

The cost of the donuts is 3 * \$68 =
 \$204 cupcakes is 2 * \$80 = \$160.
 The cost of the mini cheesecakes is 6
 * \$55 = \$330.
 cost is \$204 + \$160 + \$330 = \$694 .
 #### 694

694

Case Study 2: Dream-Instruct (7B) vs. Dream-Instruct (7B) + MetaState**Question:**

Gloria is shoe shopping when she comes across a pair of boots that fit her shoe budget. However, she has to choose between the boots and two pairs of high heels that together cost five dollars less than the boots. If one pair of heels costs \$33 and the other costs twice as much, how many dollars are the boots?

Please solve this step by step and end with #### <answer>.

Ground-truth answer: 104

Denoising Timestep $t = 0.80$

695

<i>Dream-Instruct (7B)</i>	<i>Dream-Instruct (7B) + MetaState</i>
[MASKx80] . ##### 13 [MASKx1] The answer is: 1 [MASKx2]	If one pair of heels costs \$33 [MASKx1] the [MASKx83]
Denoising Timestep $t = 0.60$	
[MASKx65] the boots [MASKx1] \$13 [MASKx1] + \$5 = \$13 [MASKx1] . ##### 13 [MASKx1] The answer is: 13 [MASKx1]	If one pair of heels costs \$33 and the other costs twice as much, then the [MASKx1] pair of heels costs [MASKx6] = [MASKx1] 66 [MASKx58] . [MASKx1]
Denoising Timestep $t = 0.40$	
[MASKx3] of [MASKx52] cost five dollars less than the boots, [MASKx1] the boots cost \$132 + \$5 = \$137 . ##### 137 The answer is: 137	If one pair of heels costs \$33 and the other costs twice as much, then the second pair of heels costs \$33 [MASKx1] 2 = \$66. [MASKx53] 104 . [MASKx1]
Denoising Timestep $t = 0.20$	
If one pair of heels costs \$33, then the other [MASKx41] heels together cost five dollars less than the boots, then the boots cost \$132 + \$5 = \$137 . ##### 137 The answer is: 137	If one pair of heels costs \$33 and the other costs twice as much, then the second pair of heels costs \$33 [MASKx1] 2 = \$66. [MASKx29] boots [MASKx1] \$99 + \$5 = \$104 . ##### 104 The answer is: 104 . [MASKx1]
Denoising Timestep $t = 0.00$	
If one pair of heels costs \$33, then the other pair costs \$33 x 2 = \$66. The total cost of the two pairs of heels is \$33 + \$66 = \$132. If the two pairs of heels together cost five dollars less than the boots, then the boots cost \$132 + \$5 = \$137 . ##### 137 The answer is: 137	If one pair of heels costs \$33 and the other costs twice as much, then the second pair of heels costs \$33 x 2 = \$66. Together, the heels cost \$33 + \$66 = \$99. The heels cost five dollars less than the boots, so the boots cost \$99 + \$5 = \$104 . ##### 104 The answer is: 104 .

696

697 A.10 Limitations

698 We discuss several limitations of the current approach. **Training overhead.** The K -step
 699 unrolling training pipeline requires $K+1$ sequential forward passes through the backbone

per training iteration (one warmup pass plus K unrolled steps), compared to a single forward pass in standard dLLM training. This multiplicative increase in computation directly raises wall-clock training time and GPU memory consumption, as intermediate activations must be retained across steps for backpropagation through the unrolled computation graph, although the recurrent modules themselves are lightweight ($\sim 0.6\%$ of backbone parameters). **Inference overhead.** At inference time, each denoising step requires executing the Mixer, Updater, and Injector in addition to the frozen backbone forward pass. Although these modules operate in compact bottleneck dimensions and add modest per-step latency, the overhead accumulates over the full denoising trajectory. Furthermore, maintaining the constant-size persistent state tensor increases peak memory usage during generation. **Potential solutions.** Several directions may mitigate the above limitations. Systems-level optimizations such as kernel fusion of the recurrent modules, hardware-aware scheduling to overlap recurrent and backbone computation, and selective recomputation strategies could reduce both training and inference overhead. Curriculum-based K scheduling or auxiliary objectives that explicitly encourage long-horizon state stability may also help close the training-to-inference extrapolation gap.